

National University of Sciences & Technology (NUST)



Instructor: Dr. Ayesha Maqbool
Parallel & Distributed Computing
CS13-E

Deliverable 2: Full System Design Document
Distributed SGD: Parameter Server & Ring AllReduce

*An Adaptive Hybrid Framework for Communication-Efficient
Distributed Training*

Name	CMS
Muhammad Jahanzeb Babar	461484
Syed Fawwad Ahmed	463933
Muhammad Shaheer Saleh	467668

Table of Contents

1	Executive Summary	3
2	Design Objectives & Scope.....	3
3	System Architecture.....	4
4	Communication Model	6
5	Consistency Model	7
6	Failure Model	8
	Activity Diagram	8
7	Performance Modeling	9
	7.1 Speedup laws	9
	7.2 Communication cost model	9
8	Scalability Assumptions	10
9	Implementation Plan	11
	9.1 Implementation stack	11
	9.2 Controller rule sketch.....	11
10	Risk Analysis	12
11	Conclusion	12
	References	13

1 Executive Summary

This system design specifies an adaptive distributed training framework for data-parallel SGD in which a cluster of workers can execute either Parameter Server (PS) synchronization or Ring AllReduce (RAR) synchronization under the control of a lightweight runtime controller. The central design idea is that no single topology is optimal across all conditions. RAR is attractive when workers are homogeneous and links are fast, because it distributes network load and avoids a centralized hotspot. PS is more operationally resilient under stragglers, uneven bandwidth, and membership changes, because centralized coordination makes recovery and partial progress easier to manage.

The proposed hybrid system therefore separates training into a data plane and a control plane. Workers perform local computation on model replicas and data shards. A controller consumes telemetry such as communication time, heartbeat delay, gradient lag, and active membership count. At safe synchronization boundaries, the controller can keep the current topology, switch to PS mode, switch to RAR mode, or enter a short recovery procedure. The design goal is not only higher throughput, but also better operational stability when the cluster becomes heterogeneous or failure-prone.

2 Design Objectives & Scope

The design is derived directly from the Assignment 1 proposal and keeps the same research question: how should a distributed SGD framework behave when it must choose between centralized and decentralized synchronization under varying runtime conditions?

Goal	Theme	Design Interpretation
O1	Comparability	Provide a single testbed in which PS, RAR, and Hybrid modes are evaluated under identical workloads and controlled perturbations.
O2	Adaptivity	Monitor runtime metrics and switch topology only when the expected communication or reliability benefit justifies the switching cost.
O3	Correctness	Preserve a single logical model state through versioning, synchronization boundaries, and checkpoint-backed recovery.
O4	Performance	Minimize synchronization overhead and improve time-to-target-accuracy rather than optimizing raw per-iteration speed alone.
O5	Resilience	Degrade gracefully under stragglers, worker crashes, and transient bandwidth reduction.

Table 1: Design Goals for the Adaptive Distributed SGD Testbed

The project scope is deliberately limited to data-parallel training on a controlled cluster with up to 16 workers, moderate-size models (logistic regression and a small CNN), and crash-stop style failures.

3 System Architecture

The hybrid framework consists of six main architectural elements (see **Figure 1**): worker processes, a parameter server service, a ring overlay for collective communication, a metrics and heartbeat monitor, an adaptive controller, and a checkpoint store. The control plane is responsible for observation and decisions; the data plane is responsible for gradient exchange and model update execution (refer **Table 2** on the next page).

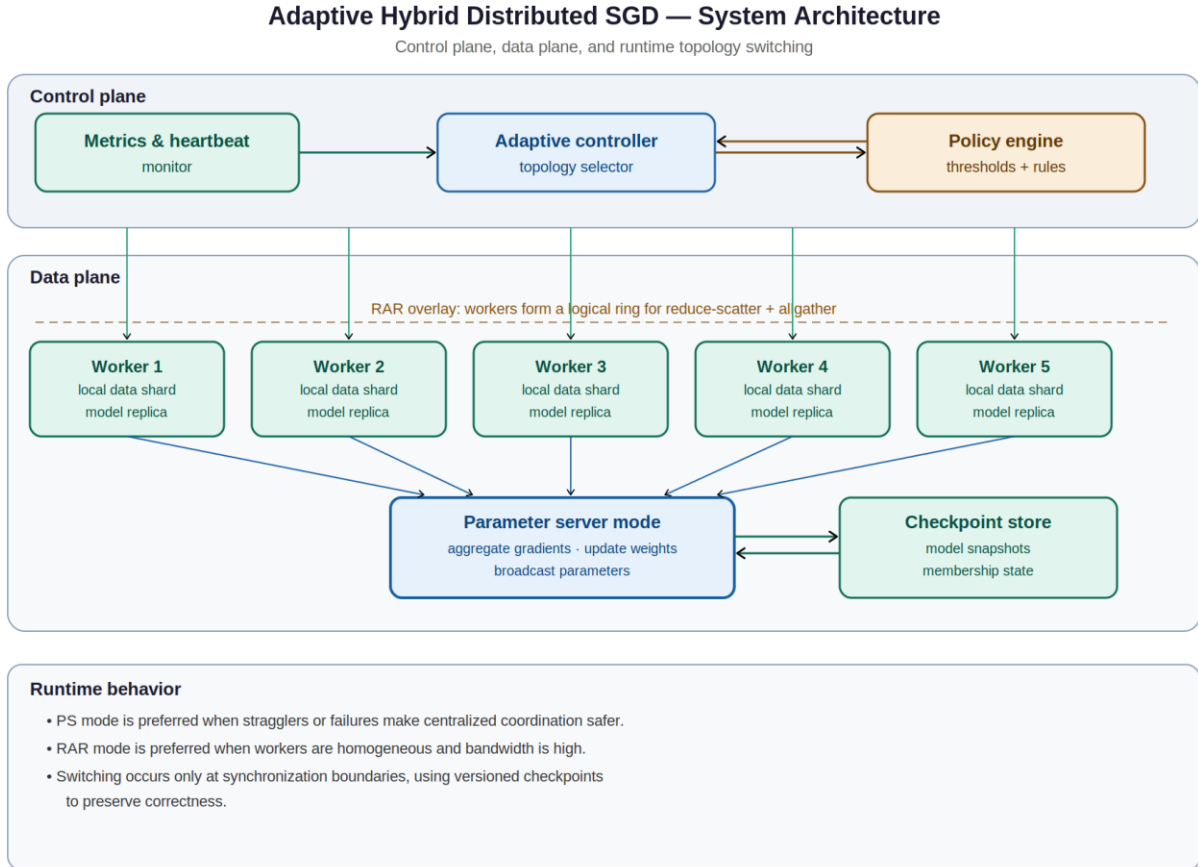


Figure 1. Architecture of the hybrid distributed SGD framework

Architectural flow:

- Each worker reads a local data shard, computes gradients, and reports telemetry after every synchronization round.
- The active synchronization engine is either PS or RAR. Only one engine is authoritative for gradient aggregation at a time.
- The controller compares observed metrics against thresholds for bandwidth saturation, straggler lag, and node liveness.

- If a topology change is justified, the system finishes the current safe round, snapshots the state, updates membership metadata, and activates the new mode.

Component	Role in the Design
Workers	Maintain local model replicas, execute forward/backward passes, compute gradients, and participate in the active synchronization topology.
Parameter Server	Aggregates pushed gradients, applies the update rule, and broadcasts the refreshed parameters. It also simplifies recovery under membership changes.
Ring Overlay	Implements reduce-scatter and allgather across the active worker set. It removes the server hotspot but requires stable ordering and stronger synchronization.
Metrics Monitor	Collects per-iteration communication time, heartbeat age, worker lag, re-transmission counts, and topology-specific error signals.
Adaptive Controller	Applies decision rules to select PS, RAR, or a recovery path. Decisions are enacted only at synchronization-safe points.
Checkpoint Store	Stores model snapshots, active membership sets, topology state, and controller metadata to support restart and mode transition correctness.

Table 2: System Components and Their Roles in the Adaptive Distributed SGD Design

4 Communication Model

The design uses a dual-plane communication model. The data plane carries gradients, model parameters, and collective messages that directly affect SGD progress. The control plane carries telemetry, heartbeat messages, membership updates, and topology decisions. Keeping these planes conceptually separate makes the runtime easier to reason about and simplifies failure recovery.

Mode	Primary Message Flow	Pattern	Operational Implication
PS mode	Worker $\rightarrow$ Server: gradient push; Server $\rightarrow$ Workers: updated weights	Centralized	Simple recovery and flexible synchronization, but server hotspot may appear.
RAR mode	Worker $\leftrightarrow$ Ring neighbors: chunk exchange for reduce-scatter/allgather	Decentralized	Bandwidth-efficient in homogeneous clusters, but sensitive to slow or failed members.
Hybrid control	Workers/monitor $\rightarrow$ Controller: metrics and heartbeats; Controller $\rightarrow$ runtime: topology command	Out-of-band control	Allows the training system to switch operating regimes without mixing inconsistent updates.

Table 3: Communication Patterns and Operational Implications by Mode

Communication procedure by mode:

1. In PS mode, each worker completes local computation, pushes its gradient to the server, waits according to the active consistency discipline, and then pulls the updated parameters.
2. In RAR mode, workers partition the gradient vector into chunks, execute reduce-scatter for partial aggregation, and then all-gather the full reduced tensor to reconstruct the next model update.
3. In Hybrid mode, workers continue using the currently active engine until the controller commits a topology change. Switching is not allowed in the middle of a round.

5 Consistency Model

Consistency is the most important correctness concern in a topology-switching SGD system. The design therefore defines consistency separately for each synchronization mode and then adds invariants for cross-mode switching (**Table 4**).

Mode / Discipline	Definition	Use in this Design
PS – BSP	Workers wait for the round to complete before continuing.	Highest consistency; most sensitive to stragglers.
PS – SSP	Workers may lag behind by at most s iterations.	Balances throughput and bounded staleness.
PS – Async	Workers do not block on every update.	Fast under heterogeneity, but may increase gradient staleness.
RAR – BSP	All workers complete the collective before advancing.	Natural fit for ring collectives; preserves aligned model versions.
Hybrid switching rule	Switch only at a synchronization boundary with a committed model version and stable membership view.	Prevents mixed-topology updates and ambiguous ownership of model state.

Table 4: Synchronization Disciplines and Their Role in the Design

Consistency in variants:

- There is exactly one authoritative model version at the end of each committed synchronization round.
- A topology switch may occur only after the current authoritative version has been checkpointed or durably recorded.
- No worker may apply gradients from one topology to parameters produced by another topology within the same round.
- The active membership set is versioned together with the model state, because correctness depends on who participated in aggregation.

This design choice means that the hybrid system is conservative at switch points even if it allows relaxed consistency inside PS mode. That conservatism is deliberate: the switching boundary is the place where ambiguity would otherwise create hard-to-debug training errors.

6 Failure Model

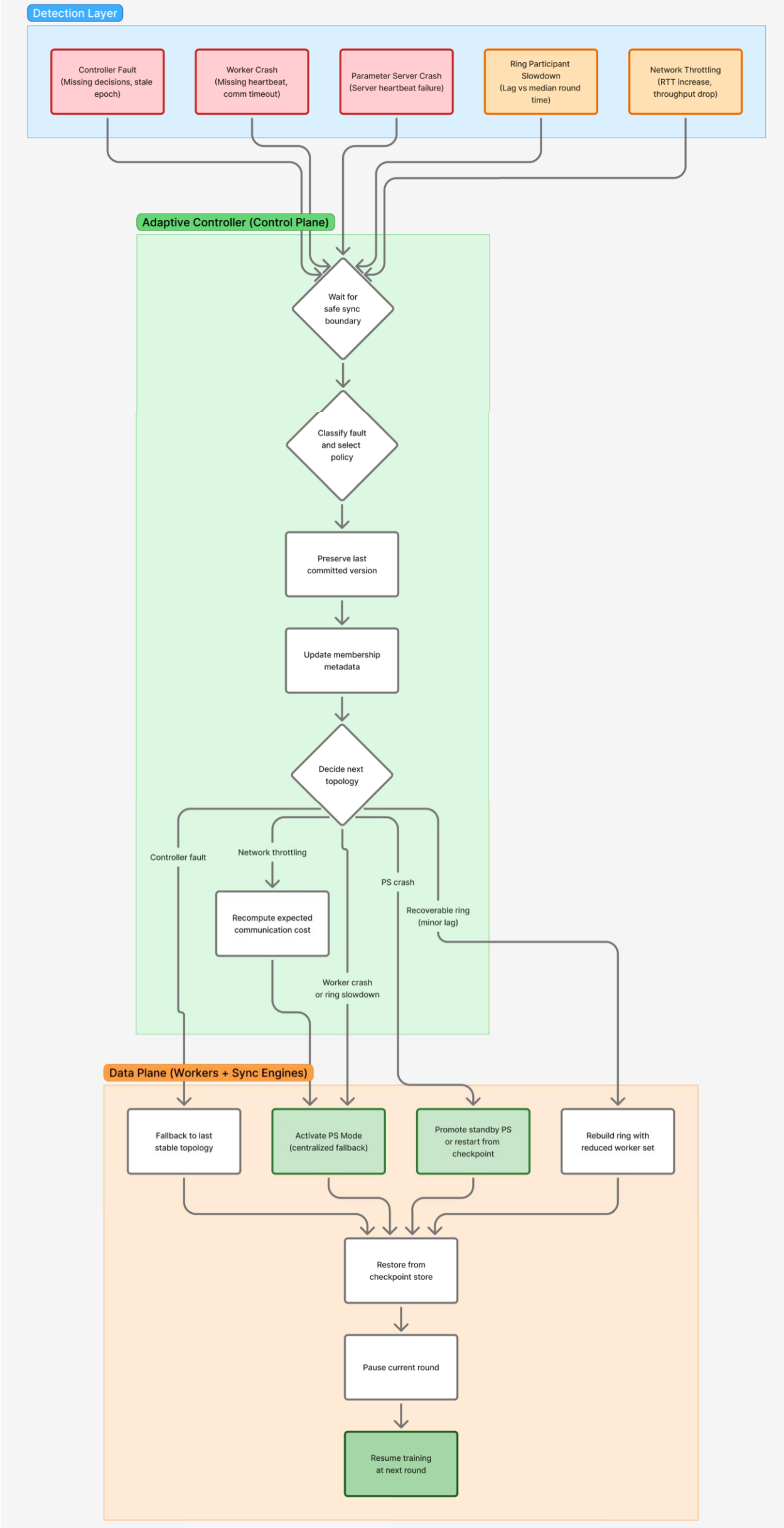
The failure model assumes crash-stop workers, transient communication slowdown, message omission or timeout, and straggler behavior caused by heterogeneous hardware or load. The design does not attempt to tolerate Byzantine workers that send malicious gradients. This bounded failure model is appropriate for a course-scale experimental implementation and still captures the most important distributed systems behaviors affecting training.

Fault	Detection	Immediate Effect	Recovery Strategy
Worker crash	Missing heartbeat and communication time-out	Loss of one participant	Pause round, update membership, restore latest committed model, continue in PS mode or rebuild ring.
Parameter server crash	Server heartbeat failure or process exit	Central coordinator unavailable	Promote standby service or restart from checkpoint; temporarily block progress to avoid split-brain.
Ring participant slowdown	Large lag relative to median round time	Entire collective is delayed	Trigger controller to switch to PS or reduce active worker set after a safe checkpoint.
Network throttling	Increased round-trip time and reduced observed throughput	Synchronization cost rises	Recompute expected cost model and prefer the topology with lower projected communication penalty.
Controller fault	Missing decisions or stale control epoch	Adaptation freezes	Fallback to last stable topology; training may continue without adaptive switching until controller restarts.

Table 5: Fault Scenarios, Detection Mechanisms, and Recovery Strategies

The most important design decision here is asymmetric recovery. Because RAR is structurally more sensitive to missing participants, the preferred recovery path after a worker failure is to checkpoint the last committed model and fall back to PS mode. This makes the hybrid design operationally safer than a static ring.

The following **Activity Diagram** clearly shows a three-lane flowchart showing how detected faults flow through the adaptive controller at safe synchronization boundaries, with most recovery paths converging on PS mode to illustrate the system's asymmetric fallback design.



7 Performance Modeling

To evaluate analytical scalability, the design uses both classical speedup laws and communication-aware cost equations. The speedup laws explain the upper bound imposed by serial overhead; the communication equations explain why the preferred topology changes as the cluster or network changes.

7.1 Speedup laws

Let α denote the serial fraction of each training iteration and n , the number of workers. The idealized upper bound under fixed problem size follows Amdahl's Law:

$$S_{Amdahl}(n) = \frac{1}{\alpha + \frac{1-\alpha}{n}}$$

When problem size scales with worker count, Gustafson's Law becomes the more relevant interpretation:

$$S_{Gustafson}(n) = n - \alpha(n - 1)$$

7.2 Communication cost model

Let M be the gradient size in bytes, B the effective bandwidth per link, and L the link latency. Ignoring optimizer-specific constant terms, the per-round synchronization cost can be approximated as follows.

Parameter Server Mode

$$T_{PS} \approx T_{comp} + \frac{2M}{B} + T_{queue} + T_{agg}$$

Ring AllReduce Mode

$$T_{RAR} \approx T_{comp} + 2(n - 1) \left[L + \frac{M}{nB} \right]$$

Hybrid Mode

$$T_{HYB} = T_{comp} + \min(E[T_{PS}], E[T_{RAR}]) + I_{switch} + T_{switch}$$

Topology	Message / Byte Trend	Latency Structure	Analytical Takeaway
Parameter Server	Per worker: $\mathcal{O}(M)$; server hotspot: $\mathcal{O}(nM)$	Two hops via server	Server bottleneck dominates as n grows or when bandwidth is uneven.
Ring AllReduce	Per worker: $\mathcal{O}(M)$	$2(n - 1)$ sequential phases	Latency chain grows with n but link load is balanced.
Hybrid	Same as active mode + switching overhead	Mode-dependent	Best choice depends on measured conditions rather than architecture alone.

Table 6: Analytical Comparison of Communication Topologies

Interpretation

RAR is usually favored when B is high and worker runtimes are tightly clustered, because the ring spreads load across links. PS becomes attractive when queueing and hotspot cost remain manageable but the cluster is heterogeneous or unstable, because centralized coordination lowers recovery complexity.

8 Scalability Assumptions

If the following listed assumptions (in **Table 7**) are violated, the expected scaling picture changes.

ID	Assumption	Reason it Matters
A1	Training is data parallel and each worker can hold the full model replica.	Model state does not exceed local memory on the project hardware.
A2	The experimental cluster uses 2 to 16 workers.	This range is enough to expose scaling trends without overwhelming implementation complexity.
A3	Bandwidth classes of 100 Mbps, 1 Gbps, and 10 Gbps can be emulated or approximated.	These conditions are sufficient to observe the PS/RAR crossover behavior.
A4	Controller decisions are lightweight compared with one training iteration.	Adaptation overhead should remain below the switching threshold budget.
A5	Failures are infrequent enough that checkpoints amortize their cost.	Checkpointing supports recovery without dominating steady-state execution.
A6	No Byzantine behavior is present.	The evaluation focuses on systems performance, not adversarial robustness.

Table 7: Design Assumptions and Their Justifications

For example, a model that no longer fits on each worker would require model parallelism or sharding, and a highly bursty network would make cost estimation much noisier. The deliverable therefore frames scalability as conditional rather than universal.

9 Implementation Plan

Implementation is divided into incremental milestones so that correctness is established before aggressive optimization or adaptive control is introduced.

Phase	Work Package	Main Tasks	Output
Phase 1	Baseline training kernel	Implement data loading, local SGD loop, logging, and deterministic seeds on a single node.	Verified single-worker baseline and reproducible metrics.
Phase 2	Static Parameter Server	Build worker/server communication with ZeroMQ; support BSP first, then optional SSP/Async variants.	Functional PS baseline with throughput and convergence logging.
Phase 3	Static Ring AllReduce	Implement collective gradient exchange with mpi4py and validate equality against PS under controlled settings.	Functional RAR baseline and correctness checks.
Phase 4	Monitoring layer	Add per-round telemetry: heartbeat age, lag ratio, communication time, active membership, and failure events.	Unified metrics substrate for analysis and control.
Phase 5	Adaptive controller	Implement threshold logic, safe switch protocol, checkpoint-backed state transition, and fallback recovery path.	End-to-end hybrid runtime.
Phase 6	Experimental campaign	Run scalability, straggler, failure, bandwidth, and model-size scenarios; collect plots and compare hypotheses.	Evaluation data for final report and presentation.

Table 8: Implementation Phases, Work Packages, and Expected Outputs

9.1 Implementation stack

Layer	Chosen Technology
Language	Python 3.10+
Parallel runtime	<code>multiprocessing</code> , <code>threading</code> , <code>mpi4py</code>
PS messaging	ZeroMQ sockets for push/pull or request/reply patterns
RAR messaging	MPI collective operations for reduce-scatter and allgather
Numerical backend	NumPy / PyTorch for model and gradient computation
Metrics & visualization	CSV / JSON logging with Matplotlib

Table 9: Technology Stack by Layer

9.2 Controller rule sketch

Decision logic

If a failure is detected, preserve the latest committed version and transition to PS mode. Else if lag ratio exceeds the straggler threshold or heartbeat skew widens, prefer PS with a relaxed consistency discipline. Else if bandwidth is healthy and lag is low, prefer RAR. Avoid switching when the expected benefit does not exceed the configured switching cost margin.

10 Risk Analysis

Risk	Likelihood	Impact	Mitigation
Topology thrashing	Medium	High	Introduce hysteresis, minimum residence time, and benefit margin before switching.
Metrics overhead distorts results	Medium	Medium	Sample coarse-grained telemetry and benchmark with instrumentation enabled/disabled.
PS becomes dominant bottleneck	High	Medium	Use bounded worker counts, batching, and efficient serialization; compare against RAR under the same load.
Ring deadlock or collective mismatch	Medium	High	Validate chunk ordering, membership updates, and barrier placement with small controlled runs first.
Checkpoint inconsistency	Low	High	Version model state and membership together; recover only from committed versions.
Experimental non-reproducibility	Medium	Medium	Use deterministic seeds, fixed workloads, and repeated runs with the same configuration.

Table 10: Risk Register with Likelihood, Impact, and Mitigation Strategies

The highest-value mitigation is disciplined state management. Many hybrid-system errors do not come from the topologies themselves but from inconsistent transitions between them. For that reason, the design gives explicit priority to versioning, checkpointing, and clear switch boundaries.

11 Conclusion

This design transforms the proposal into a concrete implementation blueprint. Its main contribution is not a claim that one topology always wins, but a disciplined systems design in which topology becomes a runtime choice informed by measurable conditions. That design aligns well with the literature review and gap analysis from Assignment 1: centralized and decentralized synchronization each have strengths, and the unresolved systems challenge is to use them adaptively while preserving correctness and analytical clarity.

References

- [1] M. Li et al., "Parameter Server for Distributed Machine Learning," Proc. NeurIPS Workshop on Big Learning, 2013.
- [2] M. Li et al., "Scaling Distributed Machine Learning with the Parameter Server," Proc. 11th USENIX OSDI, 2014.
- [3] K. Hsieh et al., "Gaia: Geo-Distributed Machine Learning Approaching LAN Speeds," Proc. 14th USENIX NSDI, 2017.
- [4] X. Lian et al., "Can Decentralized Algorithms Outperform Centralized Algorithms? A Case Study for Decentralized Parallel SGD," NeurIPS, 2017.
- [5] A. Sergeev and M. Del Balso, "Horovod: Fast and Easy Distributed Deep Learning in TensorFlow," arXiv:1802.05799, 2018.
- [6] Y. Li et al., "Pipe-SGD: A Decentralized Pipelined SGD Framework for Distributed Deep Net Training," NeurIPS, 2018.
- [7] S. Alqahtani and M. Demirbas, "Performance Analysis and Comparison of Distributed Machine Learning Systems," arXiv:1909.02061, 2019.
- [8] I. Thangakrishnan et al., "Herring: Rethinking the Parameter Server at Scale for the Cloud," SC20, 2020.
- [9] Z. Chen et al., "Rina: Enhancing Ring-AllReduce with In-Network Aggregation in Distributed Model Training," arXiv:2407.19721, 2024.
- [10] N. Provatas, I. Konstantinou, and N. Koziris, "A Survey on Parameter Server Architecture: Approaches for Optimizing Distributed Centralized Learning," IEEE Access, 2025.
- [11] N. K. Provatas, Study and Optimization of Distributed Deep Learning Under the Parameter Server Architecture, Ph.D. dissertation, NTUA, 2025.
- [12] A. Tyurin and D. Sivtsov, "Birch SGD: A Tree Graph Framework for Local and Asynchronous SGD Methods," ICLR, 2026.